INFORME PROYECTO FINAL

INTELIGENCIA ARTIFICIAL

INTEGRANTES:

NATALIA CEBALLOS POSADA

EDUARDO ANDRES PIÑEROS MANJARRES

PROFESOR:

JORGRE IVAN PADILLA BURITICA

MEDELLÍN - ANTIOQUIA

UNIVERSIDAD EAFIT

2026-2

PARTE A: Copiloto de movilidad - búsqueda de rutas con A*

1. Problema y objetivo

En el Valle de Aburrá, elegir una ruta vial puede implicar priorizar distintos criterios, como recorrer menos distancia o reducir el tiempo estimado de viaje. Este proyecto desarrolla un prototipo que encuentra una ruta entre dos puntos de la red vial de Medellín y permite comparar dos estrategias de búsqueda, A* y UCS.

El objetivo de esta primera parte es implementar A* sobre una red vial dirigida y evaluar su comportamiento frente a UCS. La aplicación permite optimizar la distancia o el tiempo estimado, visualizar la ruta y observar cuántos nodos expande cada algoritmo. Los tiempos son estimaciones basadas en atributos de la vía, el sistema no representa las condiciones actuales del tráfico.

2. Datos

La red se obtuvo de OpenStreetMap mediante OSMnx usando el tipo de red drive, por lo que representa vías utilizables por automóviles. Cada nodo incluye coordenadas geográficas, las aristas incluyen, longitud, geometría, tipo de vía y, cuando está disponible, velocidad máxima.

Para la evaluación, cada nodo representa un estado del grafo y cada arista una conexión dirigida entre dos nodos. Se utiliza la longitud de la arista, en metros, como costo cuando se optimiza distancia. Para el tiempo, se calcula el costo como longitud dividida por velocidad. Si la arista no contiene maxspeed, se asigna una velocidad estimada según la categoría vial. Por tanto, los tiempos obtenidos son aproximaciones y dependen de estos supuestos.

3. Método: A* y línea base

La implementación de A* mantiene para cada nodo el mejor costo conocido desde el origen, g(n), y prioriza en una cola de prioridad el valor f(n) = g(n) + h(n). Al alcanzar el destino, reconstruye el camino a partir de los nodos predecesores. La cola se implementa con heapq de Python.

La heurística h(n) estima el costo restante hasta el destino usando la distancia geodésica en línea recta entre las coordenadas de ambos nodos. UCS corresponde al caso de costo uniforme con h(n) = 0.

La línea base UCS/Dijkstra utiliza exactamente el mismo grafo y el mismo costo de arista que A*. De esta manera, la comparación evalúa el efecto de la heurística, en lugar de comparar funciones de costo distintas. Para cada par origen-destino se

registran el costo de la ruta, el número de nodos expandidos y el tiempo de ejecución. También se comprueba si ambos métodos obtienen el mismo costo óptimo.

4. Evaluación experimental

La evaluación se realizó con los pares origen-destino incluidos en la tabla. Se consideran dos criterios: distancia y tiempo estimado.

Origen/Destino	Criterio	Expansiones A*	Expansiones UCS	Tiempo A*	Tiempo UCS	Reduccion
309863608/14352568820	Distancia	2421	10738	15,42	29,1	77%
	Tiempo	4528	9155	27,45	29,16	51%
316069768/309028165	Distancia	667	9450	4,15	34,32	93%
	Tiempo	1502	6866	9,9	20,17	78%
316076810/316069785	Distancia	10	35	0,26	0,2	71%
	Tiempo	44	70	0,39	0,31	37%
14352568820/309026271	Distancia	1320	12005	8,01	28,96	89%
	Tiempo	2047	8650	17	26,11	76%
309863617/5512558139	Distancia	388	2404	3,8	5,42	84%
	Tiempo	1287	2977	8,35	7,81	57%

Resultados observados: A* siempre utiliza menos expansiones y menos tiempo que UCS

5. Alcance y limitaciones

La red representa calles para conducir el prototipo no incorpora congestión, incidentes, cierres, tarifas ni un modelo de emisiones. Además, cuando falta el atributo de velocidad máxima, usa valores supuestos por categoría vial. Por esta razón, la ruta por distancia se interpreta como una ruta de menor longitud en el grafo descargado, y la ruta por tiempo como una estimación calculada con las velocidades asignadas, ninguna equivale a una recomendación basada en tráfico en vivo.

PARTE 2: Agente de movilidad con RAG

La segunda parte del proyecto incorpora un asistente conversacional a la pestaña Asistente RAG del dashboard. El usuario puede preguntar sobre la medida de Pico y Placa, corredores exentos, horarios, tarifas e ingreso de bicicletas al Metro. El asistente recupera evidencia del corpus local y puede invocar una herramienta de planificación de ruta cuando la consulta también requiere calcular un trayecto.

1. Corpus y recuperación

El corpus se almacena en Datos/corpus_movilidad.json y contiene ocho fragmentos temáticos construidos a partir de páginas oficiales de la Alcaldía de Medellín y del Metro de Medellín. Cada fragmento conserva su identificador, título, entidad fuente, URL y fecha de consulta.

El módulo src/rag.py implementa recuperación léxica en español con TF-IDF. Normaliza tildes y mayúsculas, representa palabras y pares de palabras y compara la pregunta con los fragmentos mediante similitud coseno. La herramienta devuelve hasta tres resultados con sus puntajes y metadatos de fuente. El sistema inyecta esos fragmentos en el contexto del modelo para que redacte la respuesta y cite la fuente.

2. Herramientas y ciclo del agente

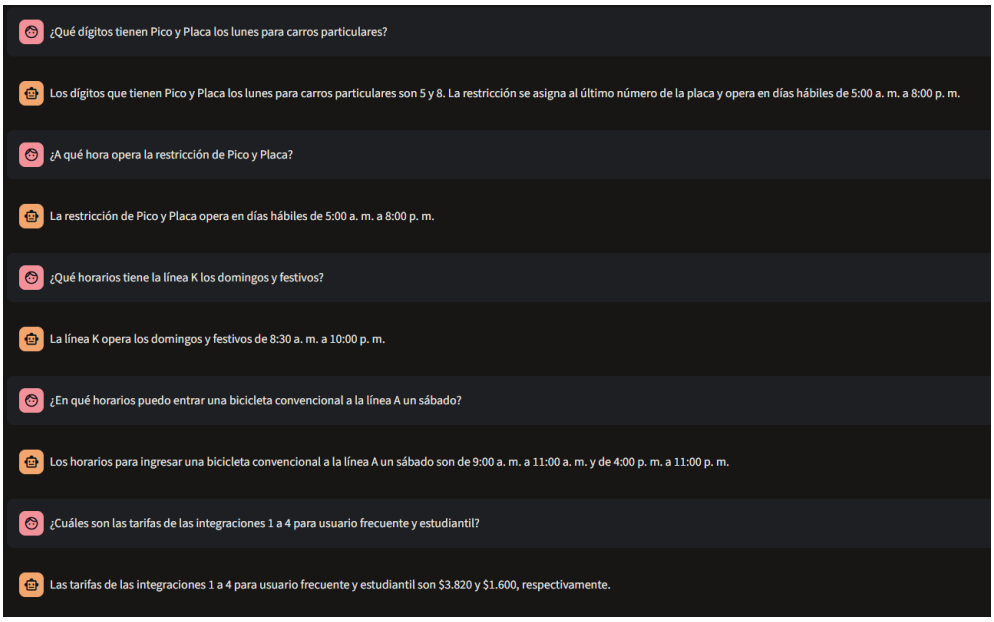
El agente expone dos herramientas mediante llamadas a funciones:

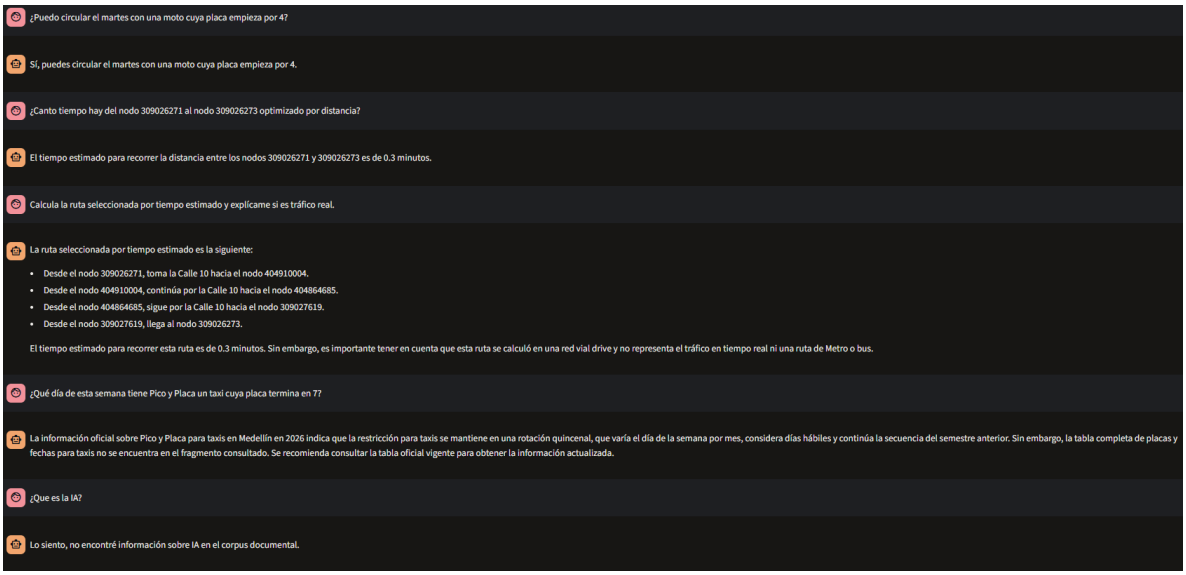
- search_mobility_docs(query) busca evidencia documental y devuelve fragmentos con título, URL y fecha de consulta. Se utiliza para preguntas factuales sobre movilidad.
- plan_route(origin, destination, metric) llama a la búsqueda de rutas implementada en la Parte 1. Devuelve distancia y tiempo estimado, cantidad de nodos de ruta, expansiones de A* y UCS y limitaciones del cálculo.

El módulo src/agent.py envía el historial y la definición de herramientas al endpoint de NVIDIA compatible con Chat Completions. Si el modelo solicita una herramienta, la aplicación ejecuta la función local correspondiente, agrega el resultado como observación y vuelve a consultar al modelo para construir la respuesta.

La integración entre ambas partes ocurre cuando una conversación necesita evidencia documental y una ruta, el agente puede consultar search_mobility_docs y, si corresponde, calcular un trayecto con plan_route. La red usada por esta última herramienta es una red vial para conducir, no calcula rutas reales de Metro o bus y no aplica automáticamente restricciones de Pico y Placa.

3. Evaluación experimental





4. Limitaciones

La recuperación por TF-IDF depende de coincidencias léxicas y trabaja con un corpus reducido. La herramienta documental solo conoce la información capturada en sus fragmentos, no consulta cambios en vivo. El asistente debe diferenciar la evidencia normativa recuperada de las estimaciones de la herramienta de rutas y señalar cuándo una pregunta supera los datos disponibles.

5. Referencias

- Alcaldía de Medellín, Secretaría de Movilidad. *Pico y Placa Medellín 2026, segundo semestre*. https://www.medellin.gov.co/es/secretaria-de-movilidad/pico-y-placa-medellin-2023-segundo-semestre/
- Metro de Medellín. *Usuarios*. https://www.metrodemedellin.gov.co/usuarios
- NVIDIA. *NVIDIA NIM API Reference: openai/gpt-oss-20b*. https://docs.api.nvidia.com/nim/reference/openai-gpt-oss-20b-infer
- Boeing, G. *OSMnx User Reference: graph_from_place*. https://osmnx.readthedocs.io/en/stable/user-reference.html
- OpenStreetMap Foundation. *Copyright and License*. https://www.openstreetmap.org/copyright